

ESTRUTURAS DE DADOS I

Exercícios Blindados: Era da IA • Arrays e Vetores em C

Aula 04: Matrizes, Vetores, Índices, Funções e rand()

Prof. Edkallenn Lima

INSTRUÇÕES DE ENTREGA

Formato: Arquivo .txt com todas as respostas em um arquivo único.

Nome do arquivo: ED1_Blindados_Arrays_SeuNome.txt

Estrutura obrigatória:

```
--- EXERCICIO 1 ---  
TAREFA 1: [resposta]  
TAREFA 2: [resposta]  
TAREFA 3: [código corrigido]  
TAREFA 4: [resposta]
```

```
--- EXERCICIO 2 ---  
... e assim por diante
```

1. Não copie respostas de IA sem auditar. Explique com suas palavras.
2. Indentação correta é obrigatória em qualquer código C entregue.
3. Cada exercício vale 2,0 pontos. Total: 10,0 pontos.
4. Entrega via Blackboard até a data limite da atividade.

Exercício 1: O Censo da Colônia Lunar

Tópico: Overflow de índice, loop com <= ao invés de <

Contexto IA: A administração da primeira colônia lunar pediu à IA um sistema para cadastrar a população dos 5 setores habitáveis. O programa compila sem erros, mas produz resultados erráticos e às vezes trava.

```
#include <stdio.h>  
#define TAM 5  
  
int main() {  
    int populacao[TAM];  
    int i, total = 0;  
  
    for (i = 0; i <= TAM; i++) {    // preenche o vetor  
        printf("Setor %d: ", i);  
        scanf("%d", &populacao[i]);  
    }  
  
    for (i = 0; i <= TAM; i++) {    // calcula total  
        total += populacao[i];  
    }  
  
    printf("Populacao total: %d\n", total);  
    return 0;  
}
```

Tarefas:

- 1) Identifique o bug: em quantas posições o vetor é acessado? Qual o índice máximo válido para `populacao[TAM]`? O que acontece quando `i == TAM`?
- 2) O slide da aula afirma que 'C não tem verificação implícita de limites em arrays'. O que isso significa na prática? Por que o compilador não lança erro nesse caso?
- 3) Corrija o código nas duas ocorrências do bug. Use `#define TAM` corretamente e explique por que a condição de parada deve ser `i < TAM` e não `i <= TAM`.
- 4) Escrever fora de um vetor em C é como entrar num prédio pelo andar que não existe, o elevador não trava, mas você pode cair. Como isso se compara ao comportamento de Java ou Python nessa situação?

Entrega: Arquivo ED1_Blindados_Arrays_SeuNome.txt — estrutura: --- EXERCICIO N --- / TAREFA 1: ... / TAREFA 2: ... / TAREFA 3: (código) / TAREFA 4: ...

Exercício 2: O Banco de Dados da SHIELD

Tópico: Inicialização de array: lixo de memória em posições não inicializadas

Contexto IA: Um agente da SHIELD pediu à IA um sistema de cadastro de IDs de agentes. A IA sabia que em C o array guarda zeros por padrão, mas esqueceu que isso só vale para variáveis globais, não locais. A soma final nunca bate.

```
#include <stdio.h>
#define MAX 10

int main() {
    int agentes[MAX];    // variavel LOCAL - nao inicializada
    int i, soma = 0;

    // IA so preencheu a primeira metade
    for (i = 0; i < 5; i++) {
        agentes[i] = (i + 1) * 1000;
    }

    // mas itera TODOS os 10 elementos
    for (i = 0; i < MAX; i++) {
        soma += agentes[i];
    }

    printf("Soma dos IDs: %d\n", soma);
    return 0;
}
```

Tarefas:

- 1) Qual o valor esperado da soma? (Calcule: $1000+2000+3000+4000+5000$.) Execute mentalmente: o que acontece nas posições `agentes[5]` a `agentes[9]`?
- 2) Explique a diferença entre variáveis globais e locais em C no que diz respeito à inicialização automática. Por que o slide alerta: 'Esquecer de inicializar os elementos de um array é um erro comum'?
- 3) Corrija de DUAS formas: (a) inicializando o array com `int agentes[MAX] = {0};` e (b) ajustando o segundo loop para iterar apenas os elementos preenchidos.

4) Que riscos práticos surgem quando se lê lixo de memória em aplicações reais como bancos de dados ou sistemas de controle? Dê um exemplo concreto.

Entrega: Arquivo ED1_Blindados_Arrays_SeuNome.txt - estrutura: --- EXERCICIO N --- / TAREFA 1: ... / TAREFA 2: ... / TAREFA 3: (código) / TAREFA 4: ...

Exercício 3: O Sistema de Temperatura da Nostromo

Tópico: *Passagem de array para função, chamada por referência simulada*

Contexto IA: A nave Nostromo precisa exibir as temperaturas originais dos setores antes e depois de um processamento. A IA gerou o código achando que a função receberia uma cópia do vetor, mas não foi o que aconteceu.

```
#include <stdio.h>
#define TAM 5

void normalizar(int v[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        v[i] = v[i] + 273; // converte Celsius para Kelvin
    }
}

int main() {
    int temp[TAM] = {-10, 20, 35, 5, -5};
    int i;

    printf("Temperaturas originais (C):\n");
    for (i = 0; i < TAM; i++)
        printf("%d ", temp[i]);

    normalizar(temp, TAM);

    // IA esperava ver os valores originais aqui
    printf("\nTemperaturas originais (C) novamente:\n");
    for (i = 0; i < TAM; i++)
        printf("%d ", temp[i]);

    return 0;
}
```

Tarefas:

- 1) O que será impresso nas duas chamadas de printf? Os valores são iguais? Por que não?
- 2) O slide afirma: 'o nome de um array é o endereço do primeiro elemento'. O que isso implica quando passamos temp para normalizar()? A função recebe uma cópia ou o original?
- 3) Corrija o programa de forma que os valores originais sejam preservados. Use uma estratégia à sua escolha: vetor auxiliar, impressão antes de chamar a função, ou outro método válido.
- 4) Compare o comportamento de arrays com o de variáveis simples (int) passadas para funções em C. Por que int x não sofre o mesmo problema que int v[]? Explique usando o conceito de pilha de chamadas.

Exercício 4: A Remoção de Tripulantes da ISS

Tópico: Função `remove_elemento`, acesso fora dos limites no `shift`

Contexto IA: A ISS precisa remover um tripulante da lista de bordo quando alguém retorna à Terra. A IA implementou a função de remoção, mas ao remover o último elemento o programa apresenta comportamento inesperado.

```
#include <stdio.h>
#define MAX 8

int remove_tripulante(int k, int v[], int n) {
    int j;
    for (j = k; j < n; j++) { // bug aqui
        v[j] = v[j + 1];      // quando j == n-1: acessa v[n]!
    }
    return n - 1;
}

int main() {
    int ids[MAX] = {101, 205, 312, 418, 520, 633, 741, 856};
    int tam = MAX;
    int i;

    printf("Lista original:\n");
    for (i = 0; i < tam; i++) printf("%d ", ids[i]);

    tam = remove_tripulante(5, ids, tam);

    printf("\nApos remover indice 5:\n");
    for (i = 0; i < tam; i++) printf("%d ", ids[i]);

    return 0;
}
```

Tarefas:

- 1) Quando `j == n-1` (última iteração do loop), qual posição é acessada na linha `v[j] = v[j+1]`? Esse índice é válido para um vetor de tamanho `n`? O que o programa lê nessa posição?
- 2) Compare com a função correta mostrada no slide da aula (`remove_elemento` com `j < n-1`). Qual a diferença exata na condição de parada e por que ela importa?
- 3) Corrija a função ajustando apenas a condição do `for`. Depois teste mentalmente com `k=5` e `n=8`: quais elementos são deslocados e qual é o vetor resultante esperado?
- 4) O slide diz que a remoção em vetor tem complexidade $O(n)$. Explique essa afirmação em termos simples: o que muda no custo se o vetor tiver 8 vs 8.000.000 elementos? Compare com o que seria em uma lista encadeada.

Exercício 5: O Gerador de Sinais do SETI

Tópico: `rand()` / `srand()`, semente fixa gerando sequência idêntica

Contexto IA: O SETI usa um sistema simulado para gerar sequências de sinais pseudo-aleatórios para treinar detectores. A IA gerou o código, mas todos os elementos do vetor sempre têm o mesmo valor, o detector recebe dados inúteis.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define TAM 10

int main() {
    int sinais[TAM];
    int i;

    for (i = 0; i < TAM; i++) {
        srand(1); // IA colocou dentro do loop
        sinais[i] = rand() % 256;
    }

    printf("Sinais capturados:\n");
    for (i = 0; i < TAM; i++)
        printf("Sinal[%d] = %d\n", i, sinais[i]);

    return 0;
}
```

Tarefas:

- 1) Execute mentalmente: se `srand(1)` é chamado a cada iteração, o que `rand()` retorna em cada chamada? Todos os `sinais[i]` terão o mesmo valor? Por quê?
- 2) Explique o que é uma 'semente' (seed) para a função `rand()`. Por que o slide recomenda `srand((unsigned)time(NULL))` chamado UMA ÚNICA VEZ antes do loop, e não dentro dele?
- 3) Corrija o código movendo `srand()` para o lugar correto. Explique também por que usar `srand(time(NULL))` gera sequências diferentes a cada execução do programa.
- 4) A função `rand()` gera números pseudo-aleatórios, não verdadeiramente aleatórios. Para aplicações críticas como simulações científicas ou criptografia, isso é suficiente? Pesquise e explique a diferença entre PRNG e TRNG em duas frases.

Entrega: Arquivo ED1_Blindados_Arrays_SeuNome.txt - estrutura: --- EXERCICIO N --- / TAREFA 1: ... / TAREFA 2: ... / TAREFA 3: (código) / TAREFA 4: ...